End-Semester Examination Study Guide: Regular Expressions, Multithreading, and Web Scraping (100 Marks)

Question 1 (20 Marks): Regular Expressions (`re` Module) & Input Pattern Validation

(a) [5 Marks] Theoretical Foundations & Core Toolkit Functions

A **Regular Expression (Regex)** is a compact mini-language used to describe, validate, locate, and manipulate structural patterns in string data. In Python, regex functionality is provided by the built-in **`re` module**.

- **`re.match(pattern, string)`** : Checks if the regex pattern matches **only at the beginning** of the string.
- **`re.search(pattern, string)`** : Scans the entire string and finds the **first occurrence** of the pattern.
- **`re.findall(pattern, string)`** : Scans the entire string and returns **all non-overlapping matches** as a list of strings.
- **`re.sub(pattern, repl, string)`** : Replaces all matching occurrences with a replacement string.
- **`re.compile(pattern)`** : Pre-compiles a regex string into a reusable pattern object to improve processing efficiency during repetitive loop iterations.

(b) [5 Marks] Pattern Vocabulary: Anchors, Character Classes, and Quantifiers

- **Anchors**: `^` asserts the start of the string; `$` asserts the end of the string.
- **Shorthand Character Classes**: `\d` matches any digit (0–9); `\w` matches any word character (letters, digits, or underscore); `\s` matches any whitespace character.
- **Character Sets**: `[abc]` matches any single character in the brackets; `[^abc]` matches any character NOT in the set.
- **Quantifiers**: `*` (0 or more), `+` (1 or more), `?` (0 or 1), `{n}` (exactly (n) repetitions).
- **Raw Strings (`r"..."`)**: Used to prevent Python string literal escaping, ensuring backslashes are passed directly to the regex engine.

© [5 Marks] Simple Baseline Solution: Email & Phone Number Validator

```
import re

# Validation logic functions using re.match
def validate_email(email_str):
    # Pattern: ^[\w.]+@[\w]+\.\w+$
    pattern = r"^[\w.]+@[\w]+\.\w+$"
    if re.match(pattern, email_str):
        return True
    return False

def validate_phone(phone_str):
    # Pattern for 10-digit Indian phone number starting with 6-9
    pattern = r"^\d{9}$"
    if re.match(pattern, phone_str):
        return True
    return False

# Execution Test
test_email = "prabu.cse@amrita.edu"
test_phone = "9876543210"

print("Email Valid:", validate_email(test_email))  # Output: True
print("Phone Valid:", validate_phone(test_phone))  # Output: True
```

(d) [5 Marks] Complex Improvement: Modular OOP Validation Engine with Custom Exception Handling

This full-mark solution introduces custom exception classes (`InvalidPatternError`), pre-compiled regex objects (`re.compile`), and structured error reporting.

```
import re

# --- Custom Exception Class ---
class InvalidPatternError(Exception):
    """Custom exception raised when input fails regex pattern validation."""
    def __init__(self, field_name, value, pattern):
        self.field_name = field_name
        self.value = value
        self.pattern = pattern
        self.message = f"Validation Failed for '{field_name}'! Value '{value}' does r
        super().__init__(self.message)
```

```
# --- Modular OOP Validator Class ---
class UserDataValidator:
    # Pre-compiled regex patterns for performance optimization
    EMAIL_REGEX = re.compile(r"^[\w.]+@[\w]+\.\w+$")
    PHONE_REGEX = re.compile(r"^\d{9}$")

    def __init__(self, email, phone):
        self.email = email
        self.phone = phone

    def validate_all(self):
        """Executes full validation and raises InvalidPatternError on failure."""
        if not UserDataValidator.EMAIL_REGEX.match(self.email):
            raise InvalidPatternError("Email", self.email, UserDataValidator.EMAIL_RE

        if not UserDataValidator.PHONE_REGEX.match(self.phone):
            raise InvalidPatternError("Phone Number", self.phone, UserDataValidator.F

        return True

# Driver Code
if __name__ == "__main__":
    records_to_test = [
        ("prabu.cse@amrita.edu", "9876543210"),  # Valid
        ("invalid-email", "9876543210"),         # Invalid Email
        ("valid@domain.com", "12345")            # Invalid Phone
    ]

    for email, phone in records_to_test:
        print(f"\n--- Testing: {email} | {phone} ---")
        try:
            validator = UserDataValidator(email, phone)
            validator.validate_all()
        except InvalidPatternError as ex:
            print(f"[Validation Error] {ex}")
        else:
            print("[Success] User data passed all pattern validation checks.")
        finally:
            print("Validation check completed.")
```

Question 2 (20 Marks): Multithreading Architecture & The Global Interpreter Lock (GIL)

(a) [5 Marks] Process vs. Thread Architecture

- **Process**: An executing instance of a program managed by the OS, possessing its own independent, isolated memory space.
- **Thread**: The smallest unit of execution inside a process. Multiple threads belonging to the same process share the same code, global data, and memory workspace, but maintain their own separate execution stacks and local registers.
- **I/O-Bound vs. CPU-Bound Tasks**: Multithreading is best suited for **I/O-bound tasks** (e.g., file reads, database queries, and web scraping) where threads spend significant time waiting for external events.

(b) [5 Marks] The Global Interpreter Lock (GIL)

- CPython enforces the **Global Interpreter Lock (GIL)**, a mutex mechanism that allows **only one thread to execute Python bytecode at any given instant**, even on a multi-core CPU.
- While the GIL prevents true parallel execution of CPU-heavy computational tasks, Python automatically releases the GIL during I/O operations (like network calls and file access), allowing other threads to run concurrently and prevent GUI or execution freezing.

© [5 Marks] Simple Baseline Solution: Parallel Execution using `threading.Thread`

```
import threading
import time

def worker_task(task_name, delay):
    print(f"Task {task_name} started...")
    time.sleep(delay)
    print(f"Task {task_name} completed after {delay}s")

# 1. Create Thread Instances
t1 = threading.Thread(target=worker_task, args=("A", 2))
t2 = threading.Thread(target=worker_task, args=("B", 1))

# 2. Launch Threads
t1.start()
t2.start()

# 3. Synchronize Execution with join()
t1.join()  # Main thread waits for t1 to complete
t2.join()  # Main thread waits for t2 to complete
```

```
print("All worker threads have finished execution.")
```

(d) [5 Marks] Complex Improvement: OOP Multithreaded Engine with Execution Profiling & Exception Handling

This improved solution subclasses `threading.Thread`, incorporates exception catching within worker tasks, and measures precise execution timing.

```
import threading
import time

# --- Subclassing Thread for OOP Encapsulation ---
class WorkerThread(threading.Thread):
    def __init__(self, task_id, duration):
        super().__init__()
        self.task_id = task_id
        self.duration = duration
        self.result = None
        self.exception = None

    def run(self):
        """Overridden method executed when start() is invoked."""
        print(f"[Thread-{self.task_id}] Execution initiated.")
        try:
            if self.duration < 0:
                raise ValueError(f"Task duration cannot be negative ({self.duration}s

            time.sleep(self.duration)
            self.result = f"Task-{self.task_id} Success"
            print(f"[Thread-{self.task_id}] Completed in {self.duration} seconds.")

        except Exception as ex:
            self.exception = ex
            print(f"[Thread-{self.task_id}] Encountered error: {ex}")

# --- Manager Engine Class ---
class ThreadBatchManager:
    def __init__(self, task_configs):
        self.task_configs = task_configs
        self.threads = []

    def execute_batch(self):
```

```
        start_time = time.perf_counter()

        # Instantiate and start worker threads
        for task_id, duration in self.task_configs:
            t = WorkerThread(task_id, duration)
            self.threads.append(t)
            t.start()

        # Wait for all worker threads to synchronize
        for t in self.threads:
            t.join()

        total_time = time.perf_counter() - start_time
        print(f"\n[Batch Complete] Processed {len(self.threads)} threads in {total_t:

# Driver Execution
if __name__ == "__main__":
    configs = [(1, 1.5), (2, 0.5), (3, -1.0)]  # Contains intentional negative durat:
    manager = ThreadBatchManager(configs)
    manager.execute_batch()
```

Question 3 (20 Marks): Thread Synchronization & Race Condition Prevention

(a) [5 Marks] Race Conditions

A **race condition** occurs when multiple threads attempt to read, modify, and write back shared global data simultaneously without proper synchronization. Because operational steps like `counter += 1` are not atomic at the bytecode level, thread context switches can overwrite concurrent updates, leading to corrupted data states.

(b) [5 Marks] Synchronization via `threading.Lock()`

- A **`Lock`** object guarantees **mutual exclusion**: only one thread can acquire the lock and execute the enclosed “critical section” of code at a time.
- Using `with lock:` automatically acquires the lock upon entering the context block and releases it upon exit, ensuring thread-safe state modification even if exceptions occur inside the block.

© [5 Marks] Simple Baseline Solution: Race Condition Demonstration & Lock Fix

```
import threading

shared_counter = 0
counter_lock = threading.Lock()

def safe_increment():
    global shared_counter
    for _ in range(100000):
        # Mutual exclusion critical section
        with counter_lock:
            shared_counter += 1

# Creating two threads targeting the same counter
t1 = threading.Thread(target=safe_increment)
t2 = threading.Thread(target=safe_increment)

t1.start()
t2.start()
t1.join()
t2.join()

print(f"Expected Count: 200000 | Actual Count: {shared_counter}")
```

(d) [5 Marks] Complex Improvement: OOP Synchronized Shared State Engine with Exception Guards

This full-mark solution builds an encapsulated thread-safe bank account system with custom exceptions (`InsufficientFundsError`), explicit lock management, and access controls.

```
import threading

# --- Custom Domain Exception ---
class InsufficientFundsError(Exception):
    def __init__(self, account_id, balance, requested):
        self.account_id = account_id
        self.balance = balance
        self.requested = requested
        super().__init__(f"Account '{account_id}': Cannot withdraw ${requested:.2f}.

# --- Thread-Safe Synchronized Bank Account Class ---
```

```
class SynchronizedBankAccount:
    def __init__(self, account_id, initial_balance):
        self.account_id = account_id
        self._balance = float(initial_balance)
        self._lock = threading.Lock()  # Private lock for thread safety

    def withdraw(self, amount):
        """Thread-safe withdrawal with lock acquisition and exception checks."""
        with self._lock:  # Critical section
            if amount <= 0:
                raise ValueError("Withdrawal amount must be greater than zero.")

            if amount > self._balance:
                raise InsufficientFundsError(self.account_id, self._balance, amount)

            self._balance -= amount
            print(f"[{threading.current_thread().name}] Withdrew ${amount:.2f} | Rema
            return self._balance

    def get_balance(self):
        with self._lock:
            return self._balance

# Concurrent Transaction Runner
def transaction_worker(account, amount):
    try:
        account.withdraw(amount)
    except (ValueError, InsufficientFundsError) as ex:
        print(f"[{threading.current_thread().name} Rejected] {ex}")

if __name__ == "__main__":
    shared_account = SynchronizedBankAccount("ACC-101", 500.0)

    # Create concurrent threads attempting withdrawals
    threads = [
        threading.Thread(target=transaction_worker, args=(shared_account, 200.0), nam
        threading.Thread(target=transaction_worker, args=(shared_account, 200.0), nam
        threading.Thread(target=transaction_worker, args=(shared_account, 200.0), nam
    ]

    for t in threads:
        t.start()
    for t in threads:
        t.join()
```

```
print(f"\nFinal Verified Account Balance: ${shared_account.get_balance():.2f}")
```

Question 4 (20 Marks): Web Scraping, Network Data Extraction, and Parsing

(a) [5 Marks] Web Scraping & Ingestion Mechanics

- **`urllib.request`** : Python standard library module used to open and fetch content from web URLs over HTTP/HTTPS.
- **HTML Parsing**: HTML documents are converted into structured tree nodes so specific tags, titles, or links can be extracted.

(b) [5 Marks] Network Exception Guardrails

When performing web scraping, network requests can fail due to invalid URLs, broken endpoints, or missing connection handles. Exception handling with `urllib.error.URLError` and `HTTPError` prevents script crashes.

© [5 Marks] Simple Baseline Solution: URL Ingestion & Hyperlink Extractor

```
import urllib.request
import re

def scrape_website_title(url):
    try:
        # 1. Fetch Web Page Content
        response = urllib.request.urlopen(url)
        html_bytes = response.read()
        html_text = html_bytes.decode('utf-8', errors='ignore')

        # 2. Extract <title> using Regex Pattern
        title_pattern = r"<title>(.*?)</title>"
        match = re.search(title_pattern, html_text, re.IGNORECASE)

        if match:
            return match.group(1)
        return "No Title Found"

    except Exception as e:
```

```
        return f"Error: {e}"

# Execution
target_url = "[suspicious link removed]"
print("Extracted Title:", scrape_website_title(target_url))
```

(d) [5 Marks] Complex Improvement: OOP Web Scraper with Regex Link Mining and Network Error Handling

This full-mark solution builds a modular web page scraper that extracts all hyperlinks (`<a href="...">`) using regular expressions while handling network failures safely.

```
import urllib.request
import urllib.error
import re

# --- Custom Web Scraper Class ---
class WebDataExtractor:
    TITLE_REGEX = re.compile(r"<title>(.*?)</title>", re.IGNORECASE)
    HREF_REGEX = re.compile(r'href=["\'](https?://[^\s"\']+)["\']', re.IGNORECASE)

    def __init__(self, target_url):
        self.target_url = target_url
        self.raw_html = ""
        self.extracted_title = ""
        self.extracted_links = []

    def fetch_page(self):
        """Downloads web page bytes and decodes string data safely."""
        req = urllib.request.Request(
            self.target_url,
            headers={'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)'}
        )

        try:
            with urllib.request.urlopen(req, timeout=5) as response:
                if response.status != 200:
                    raise urllib.error.HTTPError(self.target_url, response.status, "H

                bytes_data = response.read()
                self.raw_html = bytes_data.decode('utf-8', errors='ignore')
```

```
            except urllib.error.HTTPError as ex:
                raise RuntimeError(f"HTTP Server Error ({ex.code}): {ex.reason}")
            except urllib.error.URLError as ex:
                raise RuntimeError(f"Network Connection Error: {ex.reason}")

        def parse_data(self):
            """Extracts page title and outgoing links using compiled regex patterns."""
            if not self.raw_html:
                raise ValueError("No HTML content available. Invoke fetch_page() first.")

            # Extract Title
            title_match = WebDataExtractor.TITLE_REGEX.search(self.raw_html)
            self.extracted_title = title_match.group(1).strip() if title_match else "Unt

            # Extract Outgoing HTTP Links
            self.extracted_links = WebDataExtractor.HREF_REGEX.findall(self.raw_html)

        def get_summary(self):
            return {
                "url": self.target_url,
                "title": self.extracted_title,
                "link_count": len(self.extracted_links),
                "sample_links": self.extracted_links[:3]  # Return top 3 links
            }

    # Driver Routine
    if __name__ == "__main__":
        scraper = WebDataExtractor("[suspicious link removed]")

        try:
            print(f"Fetching page: {scraper.target_url}...")
            scraper.fetch_page()
            scraper.parse_data()
            summary = scraper.get_summary()

        except RuntimeError as ex:
            print(f"[Network Failure] {ex}")
        except ValueError as ex:
            print(f"[Parsing Failure] {ex}")
        else:
            print("\n--- Scraping Summary Report ---")
            print(f"Page Title : {summary['title']}")
            print(f"Links Found: {summary['link_count']}")
            print("Sample Links:")
            for link in summary['sample_links']:
                print(f" - {link}")
```

```
    finally:
        print("Scraping execution finished.")
```

Question 5 (20 Marks): Capstone Integration — Multithreaded GUI Web Scraper & Regex Validator

(a) [5 Marks] Non-Blocking GUI Threading Architecture

Running long-running I/O operations (such as web scraping or pattern matching over large datasets) directly on Tkinter's main execution thread causes the user interface to freeze and become unresponsive. Offloading background network tasks to a secondary worker thread keeps Tkinter's `mainloop()` responsive.

(b) [10 Marks & © [5 Marks] Complete Capstone Application: Multithreaded GUI Scraper with Tkinter & Regex

This continuous, end-to-end OOP application combines `Tkinter` widgets (`Entry`, `Listbox`, `Frame`, `messagebox`), background `threading.Thread` execution, `urllib` network calls, and `re` pattern validation.

```
import tkinter as tk
from tkinter import messagebox
import urllib.request
import urllib.error
import re
import threading

# --- Custom Exception Class ---
class InvalidURLError(Exception):
    def __init__(self, url):
        super().__init__(f"Invalid URL Format: '{url}'. Must start with 'http://' or

# --- Multithreaded GUI Application Class ---
class MultithreadedScraperApp(tk.Tk):
    # Regex to validate HTTP/HTTPS URL format
    URL_VALIDATION_REGEX = re.compile(r"^https?://[^\s/$.?#].[^\s]*$", re.IGNORECASE)

    # Regex to extract links from scraped HTML
    LINK_REGEX = re.compile(r'href=["\'](https?://[^\s"\']+)["\']', re.IGNORECASE)
```

```
        def __init__(self):
            super().__init__()
            self.title("Multithreaded Web Link Scraper Studio")
            self.geometry("550x420")
            self.config(bg="#f2f4f4")

            self.build_ui()

        def build_ui(self):
            # 1. Header Frame
            header = tk.Frame(self, bg="#2c3e50", pady=10)
            header.pack(side=tk.TOP, fill=tk.X)

            tk.Label(
                header,
                text="Multithreaded Web Scraper Studio",
                font=("Arial", 14, "bold"),
                fg="white",
                bg="#2c3e50"
            ).pack()

            # 2. Input Form Panel
            form_frame = tk.LabelFrame(self, text=" Target Web Page ", font=("Arial", 10,
            form_frame.pack(pady=10, fill=tk.X, padx=15)

            tk.Label(form_frame, text="URL:", bg="#f2f4f4").grid(row=0, column=0, sticky=

            self.ent_url = tk.Entry(form_frame, width=32, font=("Arial", 10))
            self.ent_url.insert(0, "[suspicious link removed]")
            self.ent_url.grid(row=0, column=1, padx=5)

            self.btn_scrape = tk.Button(
                form_frame,
                text="Scrape Links",
                command=self.start_scraping_thread,
                bg="#27ae60",
                fg="white",
                font=("Arial", 9, "bold")
            )
            self.btn_scrape.grid(row=0, column=2, padx=10)

            # 3. Output Display Area (Listbox + Scrollbar Frame)
            display_frame = tk.Frame(self, bg="#f2f4f4")
            display_frame.pack(pady=5, fill=tk.BOTH, expand=True, padx=15)

            self.scrollbar = tk.Scrollbar(display_frame, orient=tk.VERTICAL)
```

```
        self.link_listbox = tk.Listbox(
            display_frame,
            font=("Courier", 9),
            selectmode=tk.SINGLE,
            yscrollcommand=self.scrollbar.set
        )
        self.scrollbar.config(command=self.link_listbox.yview)

        self.link_listbox.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
        self.scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

        # 4. Status Bar
        self.status_var = tk.StringVar(value="Status: Ready")
        self.status_bar = tk.Label(self, textvariable=self.status_var, bd=1, relief='
        self.status_bar.pack(side=tk.BOTTOM, fill=tk.X)

    def start_scraping_thread(self):
        """Validates URL input and spawns a background worker thread."""
        target_url = self.ent_url.get().strip()

        try:
            # Validate URL with Regex
            if not MultithreadedScraperApp.URL_VALIDATION_REGEX.match(target_url):
                raise InvalidURLError(target_url)

            # Disable button and update status during processing
            self.btn_scrape.config(state=tk.DISABLED)
            self.status_var.set("Status: Scraping in background thread...")
            self.link_listbox.delete(0, tk.END)

            # Spawn worker thread to avoid blocking mainloop
            worker = threading.Thread(target=self.bg_scrape_worker, args=(target_url,
            worker.start()

        except InvalidURLError as ex:
            messagebox.showwarning("URL Format Error", str(ex))
            self.btn_scrape.config(state=tk.NORMAL)
            self.status_var.set("Status: Validation Error")

    def bg_scrape_worker(self, url):
        """Background thread worker for network request and parsing."""
        try:
            req = urllib.request.Request(url, headers={'User-Agent': 'Mozilla/5.0'})
            with urllib.request.urlopen(req, timeout=6) as response:
                html_text = response.read().decode('utf-8', errors='ignore')
```

```
            # Extract links using Regex
            links = MultithreadedScraperApp.LINK_REGEX.findall(html_text)

            # Update GUI safely back on main thread context
            self.after(0, self.update_gui_success, links)

        except Exception as ex:
            # Pass exception message back to main thread
            self.after(0, self.update_gui_failure, str(ex))

    def update_gui_success(self, links):
        """Callback executed on main thread when scraping succeeds."""
        self.btn_scrape.config(state=tk.NORMAL)
        self.status_var.set(f"Status: Complete! Found {len(links)} links.")

        if not links:
            self.link_listbox.insert(tk.END, "No links found on target page.")
            return

        for idx, link in enumerate(links, start=1):
            self.link_listbox.insert(tk.END, f"{idx:02d}. {link}")

        messagebox.showinfo("Scraping Complete", f"Successfully extracted {len(links)

    def update_gui_failure(self, error_message):
        """Callback executed on main thread when scraping fails."""
        self.btn_scrape.config(state=tk.NORMAL)
        self.status_var.set("Status: Scraping Failed")
        messagebox.showerror("Network Scraping Error", f"Failed to fetch target URL:\

if __name__ == "__main__":
    app = MultithreadedScraperApp()
    app.mainloop()
```

Key Exam Takeaways Summary Table

Concept	Primary Module / Keyword	Essential Exam Rules & Functions

Regular Expressions

```
import re
```

`re.match()` (start only), `re.search()` (first match), `re.findall()` (all matches list), `re.sub()` (replace).

Multithreading

```
import threading
```

`Thread(target=f, args=())`, `start()`, `join()` (synchronize execution). GIL restricts true CPU parallelism.

Thread Locking

```
threading.Lock()
```

Prevents race conditions. Use `with lock:` context manager for atomic critical section execution.

Web Scraping

```
urllib.request
```

`urlopen(url).read().decode()` fetches web pages. Always catch `URLError / HTTPError.`

GUI Threading

```
app.after(ms, func)
```

Run I/O scraping inside background threads; update Tkinter UI elements via `after()` or main thread callbacks.